Enki Authorization Redesign

Design of record — capability matrix, policies, controllers, cleanup, tests

Mike King + Claude Opus 4.7

2026-05-01

1. Goals

- Deterministic, table-driven authorization for every controller action.
- Single parametric requirement, single handler — exhaustively testable matrix.
- Capability claims as orthogonal grants (start with **licensing**).
- Lock down the admin area, especially the master user roster.
- Clean up the unused **TenantUser.Role** model now that it isn't gating anything.
- Tenant users keep using existing Team controllers in the interim (dedicated controllers come later); restricted to Field-equivalent.

2. Axes

Five orthogonal axes drive every authorization decision. Conflating any pair of these is where past designs went wrong.

Axis	Storage	Purpose
IsEnkiAdmin	ApplicationUser column	Cross-tenant root bypass
UserType (Team / Tenant)	ApplicationUser column	Which surface area you can reach
TeamSubtype (Field / Office / Supervisor)	ApplicationUser column	What VERBS you can perform
enki:capability claims	AspNetUserClaims rows	Atomic grants for specific operations
TenantUser membership	master.TenantUsers row	SCOPE — which tenants you can touch

> **TenantUser.Role** is RETIRED — column drop, enum drop, migration.

3. Capability matrix

`A` = Admin (`IsEnkiAdmin=true`). `+L` = `enki:capability=licensing` claim. `T` = Tenant-type user (interim behavior; goes away when dedicated portal lands).

Capability	Field	Office	Sup	A	+L	T
Read tenant data, master Tools, master Calibrations	✓	✓	✓	✓		✓ (bound)
Add / edit / use Runs, Shots, bin uploads	✓	✓	✓	✓		✓ (bound)
CRUD Jobs / Wells / TieOns / Surveys / Tubulars / Formations / CommonMeasures / Magnetics / Logs / Comments		✓	✓	✓		
Edit existing tenant settings (display name / notes / contact email)		✓	✓	✓		
CRUD master Calibrations		✓	✓	✓		
Run computation pipelines (CalibrationProcessing)		✓	✓	✓		
Provision Tenant-type users; edit / lock / unlock / reset-pwd / session-lifetime on Tenant-type users		✓	✓	✓		
CRUD master Tools			✓	✓		
Provision new tenants			✓	✓		
Deactivate / Reactivate / Archive tenant			✓	✓		
Add / remove tenant members			✓	✓		
Read master roster (user picker)			✓	✓		
Generate / Revoke licenses			✓	✓	✓	
Provision Team-type users + every admin op on Team-type users (lock / reset-pwd / session-lifetime / IsEnkiAdmin / capability grants)				✓		

Two patterns to call out:

- **AdminUsersController actions** decide based on the **target user's UserType** — Office can do anything to a Tenant user, only Admin can touch a Team user. Implementation: a `RequireSufficientAuthorityFor(target)` helper called by each action.
- **`IsEnkiAdmin` and capability grants** are always Admin-only regardless of target type — they are permission-bestowing operations.

4. Policy catalog

12 named policies, 1 requirement type.

```
"EnkiApiScope"               // any signed-in caller
"CanAccessTenant"            // tenant member or admin
                             //  + Tenant-type user with matching tenant_id (interim)
"CanWriteTenantContent"      // Office+ tenant member, or admin
"CanDeleteTenantContent"     // Office+ tenant member, or admin (separate name; future
tightening)
"CanWriteMasterContent"      // Office+ or admin
"CanDeleteMasterContent"     // Office+ or admin
"CanManageMasterTools"       // Supervisor+ or admin (Tools all CRUD)
"CanProvisionTenants"        // Supervisor+ or admin
"CanManageTenantLifecycle"   // Supervisor+ or admin (deactivate / reactivate / archive)
"CanManageTenantMembers"     // Supervisor+ tenant member, or admin (was: Role==Admin)
"CanReadMasterRoster"        // Supervisor+ or admin (user picker, was EnkiApiScope)
"CanManageLicensing"         // Supervisor+ OR licensing capability, or admin
"EnkiAdminOnly"              // admin only
```

Requirement (single parametric type):

```
public sealed record TeamAuthRequirement(
    TeamSubtype? MinimumSubtype     = null,
    string?      GrantingCapability = null,
    bool         TenantScoped       = false,
    bool         RequireAdmin       = false
) : IAuthorizationRequirement;
```

5. Handler decision tree

One handler. Logic in the strict order below; first matching rule wins.

```
1. If R.RequireAdmin:
       succeed iff P.IsEnkiAdmin  ->  return

2. If P.IsEnkiAdmin:
       succeed  ->  return                              (root bypass)

3. If P.user_type == Tenant:                            (interim Tenant-user path)
       if not R.TenantScoped:                deny       (master ops never reach Tenant users)
       resolve P.tenant_id -> boundCode (cached)
       if boundCode != route.tenantCode:     deny
       if R.MinimumSubtype is null:          succeed    (Field-equivalent ops only)
       else:                                 deny
       return

4. If R.TenantScoped:                                   (Team path, tenant op)
       check TenantUser membership of route.tenantCode (cached, existing logic)
       if not a member:                      deny

5. If R.MinimumSubtype is null:               succeed   (any signed-in member, no verb gate)

6. If P.team_subtype >= R.MinimumSubtype:    succeed

7. If R.GrantingCapability != null AND
   P has claim "enki:capability"=R.GrantingCapability:
                                              succeed

8.                                            deny
```

Edge cases verified:

- Team user with no `team_subtype` claim (data drift) → fails step 6, fails step 7 → deny. Fail-safe.
- Tenant user trying master endpoint → step 3 deny.
- Tenant user trying Office-tier tenant endpoint → step 3 deny on `MinimumSubtype != null`.
- Office user with Licensing capability hitting `/licenses` → step 6 fails (Office < Sup), step 7 succeeds.
- Tenant user already cannot be admin (validator); seed reconciler would throw at boot.

6. Controller mapping (every controller, every action)

6.1 Tenant-scoped (WebApi)

Controller	Action	Today	New
`JobsController`	GET	CanAccessTenant	CanAccessTenant
	POST / PATCH	CanAccessTenant	CanWriteTenantContent
	DELETE	CanAccessTenant	CanDeleteTenantContent
`WellsController`, `TieOnsController`, `SurveysController`, `TubularsController`, `FormationsController`, `CommonMeasuresController`, `MagneticsController`, `LogsController`, `CommentsController`	(same shape as Jobs)	CanAccessTenant	same as Jobs
`RunsController`	GET / POST / PATCH / DELETE	CanAccessTenant	CanAccessTenant
`ShotsController`	(same as Runs)	CanAccessTenant	CanAccessTenant
`AuditController`	GET (tenant audit)	CanAccessTenant	CanAccessTenant
`TenantMembersController`	GET	CanAccessTenant	CanAccessTenant
	POST / DELETE	CanManageTenantMembers (Role==Admin)	CanManageTenantMembers (now Sup+)
	PATCH /role	present	**DELETED in cleanup commit**

6.2 Master-scoped (WebApi)

Controller	Action	Today	New
`ToolsController`	GET (read)	EnkiApiScope	EnkiApiScope
	POST / PATCH / DELETE	EnkiApiScope (!)	CanManageMasterTools
`CalibrationsController`	GET	EnkiApiScope	EnkiApiScope
	POST / PATCH	EnkiApiScope	CanWriteMasterContent
	DELETE	EnkiApiScope	CanDeleteMasterContent
`CalibrationProcessingController`	POST (compute)	various	CanWriteMasterContent
`MasterUsersController`	GET (picker)	EnkiApiScope	CanReadMasterRoster
	POST /sync	EnkiAdminOnly	CanWriteMasterContent
`TenantsController`	GET (list / detail)	EnkiApiScope	EnkiApiScope (filtered internally)
	POST (provision)	EnkiAdminOnly	CanProvisionTenants
	POST /deactivate, /reactivate, /archive	EnkiAdminOnly	CanManageTenantLifecycle
	PATCH (settings)	EnkiAdminOnly	CanWriteMasterContent
`LicensesController`	GET / POST / Revoke	EnkiAdminOnly	CanManageLicensing
`MasterAuditController`	GET	EnkiAdminOnly	EnkiAdminOnly
`SystemSettingsController`	all	EnkiAdminOnly	EnkiAdminOnly

6.3 Identity host

Controller	Action	Today	New
`AdminUsersController`	class-level `[Authorize]`	EnkiAdminOnly	**DROPPED** — per-action attributes
	GET (list / detail)	(admin)	EnkiAdminOnly
	POST (Create)	(admin)	inner check: Team→EnkiAdminOnly, Tenant→CanWriteMasterContent
	PUT (Update)	(admin)	per-target check (helper)
	POST /lock, /unlock, /reset-password, /session-lifetime	(admin)	per-target check (helper)
	POST /admin (toggle IsEnkiAdmin)	(admin)	EnkiAdminOnly (capability-bestowing)
	POST /{id}/capabilities/{cap} (NEW)	—	EnkiAdminOnly
	DELETE /{id}/capabilities/{cap} (NEW)	—	EnkiAdminOnly
`AuthEventsController`, `IdentityAuditController`	reads	EnkiAdminOnly	EnkiAdminOnly
`MeController`, `AuthorizationController`	self / OIDC	unchanged	unchanged

Per-target check helper

```
// Returns null if caller can act on this target; else 403 ProblemDetails.
private IActionResult? RequireSufficientAuthorityFor(ApplicationUser target)
{
    if (User.IsEnkiAdmin())                                  return null;  // admin bypass
    if (target.UserType == UserType.Tenant
        && User.HasTeamSubtypeAtLeast(TeamSubtype.Office))      return null;  // Office can
         manage Tenant users
    return Forbid();
}
```

7. TenantUser.Role cleanup

Real chunk of refactoring. Worth its own commit.

- **Entity**: drop the `Role` property on `TenantUser`.
- **Enum**: delete `TenantUserRole` (file + namespace usage).

- **Migration**: drop the `Role` column on `master.TenantUsers`.
- **Handler**: `CanManageTenantMembersHandler` no longer queries Role; replaced by subtype-based check (the new `TeamAuthHandler`).
- **DTOs in `Shared.Tenants.TenantMemberDtos.cs`**:
  - `TenantMemberDto.Role` → drop
  - `AddTenantMemberDto.Role` → drop (just `UserId`)
  - `SetTenantMemberRoleDto` → delete record
- **Controller**: `TenantMembersController` — drop the role-set action (PATCH /role); Add becomes parameterless body.
- **Blazor**: any UI that displays or edits Role on members — drop those columns / fields.
- **Seed**: any TenantUser seed that sets Role — strip the assignment.
- **Tests**: anywhere a test sets `Role = Admin` to satisfy `CanManageTenantMembers` → swap for granting Supervisor subtype to the principal.

8. Capability claims infrastructure

Constants (Shared)

```
public static class EnkiCapabilities
{
    public const string Licensing = "licensing";
    // future capabilities go here
}

public static class EnkiClaimTypes
{
    public const string Capability = "enki:capability";
}
```

Storage

`AspNetUserClaims` rows. Type = `EnkiClaimTypes.Capability`, value = capability constant. **No new table, no migration.**

Emission

ASP.NET Identity's base `UserClaimsPrincipalFactory.CreateAsync` already pulls AspNetUserClaims onto the principal. Add an explicit case in `AuthorizationController.GetDestinations` so capability claims land on AccessToken + IdentityToken (default destination is AccessToken-only for unknown types; matching for clarity).

Admin endpoints

- `POST /admin/users/{id}/capabilities/{capability}` — idempotent grant; if already present 204 no-op; otherwise add + rotate stamp + audit.
- `DELETE /admin/users/{id}/capabilities/{capability}` — idempotent revoke; if absent 204 no-op; otherwise remove + rotate stamp + audit.

- Both `EnkiAdminOnly`. Audited as `CapabilityGranted` / `CapabilityRevoked` with capability name in `NewValues` JSON.

Validator update

`UserClassificationValidator` rejects capability grants on Tenant-type users (capabilities are conceptually a Team-side construct). Add a `Validate(target, capability)` overload, or check in the endpoint.

DTO update

`AdminUserDetailDto` carries a `Capabilities` string array so the UI can render the checkboxes.

Seed update

`SeedUser` gets a `Capabilities` list field (default empty). `IdentitySeedData` reconciles claims on every boot — add/remove rows to converge.

9. Master roster lock-down

Beyond just changing `MasterUsersController.GET` to `CanReadMasterRoster`:

- **Filter the response** — don't return Tenant-type users in the picker (they can't be added as Team members of other tenants anyway). Reduces the surface even for legitimate callers.
- **Return shape stays minimal** — `{Id, IdentityId, Username}`; no email, no admin flag, no classification leaked.
- **Audit each fetch** via the existing audit infrastructure (one row per call).

10. Tenant user interim behavior

- `CanAccessTenantHandler` keeps the Tenant-user clause from the prior commit (matches `tenant_id` claim against route).
- `TeamAuthHandler` step 3 explicitly handles Tenant users with the same logic — succeed for `MinimumSubtype is null`, deny otherwise.
- Tenant users CAN hit: `JobsController.GET`, `RunsController.*` (all CRUD), `ShotsController.*` (all CRUD), `LogsController.GET`, etc.
- Tenant users CAN'T hit: master endpoints, write/delete on Jobs / Wells / etc. (no subtype claim → deny).
- When the dedicated portal lands, the interim path goes away in one diff: remove the Tenant clause from both handlers, route Tenant users to portal controllers.

11. Module visibility (Blazor)

`ModuleVisibilityService` reads `team_subtype` and capability claims off the cookie principal. Visibility is convenience; the policy gate is the actual enforcement.

Nav item	Visible to
Master Tenants list	Everyone Team (read)
Master Tools menu	Field-and-up read; Sup+ for write actions in the page
Master Calibrations menu	Field-and-up read; Office+ for write actions
Master Licenses menu	Sup+ OR Licensing capability OR admin
Master Tenant provisioning button	Sup+ OR admin
Tenant settings edit button	Office+ OR admin
Tenant deactivate / archive buttons	Sup+ OR admin
Tenant member management	Sup+ OR admin
Identity admin (`/admin/users`)	Admin only
Tenant user nav (interim)	Just `/tenants/{boundCode}/jobs` (or similar landing) — no master nav

12. Migration / schema changes

- **`RemoveTenantUserRole`** (master DB) — drop `Role` column on `TenantUsers`.
- Identity DB — no schema change. Capabilities reuse `AspNetUserClaims`.
- Existing migrations from prior commits stay as-is (additive, already applied).

13. Test strategy

Handler matrix

`TeamAuthHandlerTests.cs` — parameterised `[Theory]` with one case per (subtype × policy × is_admin × user_type × has_capability × is_member). ~80 combinations. Covers every step in the decision tree.

Controller smoke (one positive + one negative per tier change)

- Field on `JobsController.POST` → 403
- Office on `JobsController.POST` → 200
- Office on `LicensesController.POST` → 403
- Office on `LicensesController.POST` **with Licensing capability** → 200
- Office on `ToolsController.POST` → 403
- Sup on `ToolsController.POST` → 200
- Office on `TenantsController.POST` (provision) → 403
- Sup on `TenantsController.POST` → 200
- Tenant user on `JobsController.POST` (their bound tenant) → 403
- Tenant user on `JobsController.GET` (their bound tenant) → 200
- Tenant user on `JobsController.GET` (different tenant) → 403
- Tenant user on `LicensesController.GET` → 403

Capability tests (grant / revoke endpoint)

- Grant happy path → claim present, audit row written, stamp rotated.
- Revoke happy path.
- Idempotent: grant twice = 204 + no-op.
- Grant on Tenant user → 400 (validator rejects).

Per-target check tests (AdminUsersController)

- Office editing a Tenant user → 200.
- Office editing a Team user → 403.
- Office locking a Tenant user → 200.
- Office calling `/admin` (IsEnkiAdmin toggle) → 403 always.

TenantUser.Role cleanup regression

Existing tests that set `Role = Admin` to satisfy `CanManageTenantMembers` → migrated to grant Supervisor subtype on the principal.

Estimated test churn

Survey pass during implementation will give a real number. Ballpark: 30–50 existing tests need fixture updates (add `team_subtype` claim), ~60 new tests for the matrix + capability + per-target.

14. Implementation order

Four commits to keep review tractable. Each builds + tests green on its own.

#	Commit	Scope
1	**Cleanup commit**	Drop `TenantUser.Role`: entity, enum, migration, DTOs, controller, Blazor, tests. Pure refactor; no behavior change in production paths.
2	**Capability infrastructure commit**	Constants, claim type, admin endpoints, claim emission destinations, validator update, DTO/seed support, UI checkbox.
3	**Authorization redesign commit**	`TeamAuthRequirement` + `TeamAuthHandler` + 12 policies in `Program.cs` + per-controller `[Authorize(Policy=...)]` retag + `AdminUsersController` per-target restructure + `MasterUsersController.GET` lock-down + master roster filter.
4	**Blazor visibility + tests commit**	`ModuleVisibilityService` updates + handler matrix tests + controller smoke + capability tests + per-target tests + fixture updates for the test churn.

15. Open questions

Only what we don't yet have a default for. Everything else above is locked.

1. Tools READ gating

Reading the response *"no need for the additional granularity"* as **leave Tools READ open** so Field engineers can identify what tool they're operating on a Run form. Confirm? Alternative is Tools fully Supervisor+ including read; that would mean Run create flow needs a different lookup path for Field.

2. Capability validator on Tenant users

Should `UserClassificationValidator` reject capability grants on Tenant users explicitly, or rely on the fact that no current capability (`licensing`) makes sense for them? **Recommendation: reject explicitly** (defense in depth + clear error message).

End of document.